Luxair interview terms

Tomor emlekezteto azokhoz a fogalmakhoz, amikre rakerdeztel. Telefonon gyors atfutasra, nem tankonyvnek.

1. Traffic, performance, scaling

Fogalom	Emlekezteto
100M request/month	Kb. 39 rps atlag. Sizinghoz peak, request ara, DB/downstream limit, p95/p99 kell.
Throughput	Mennyi munkat fejez be idoegyseg alatt. API-nal: completed requests/sec.
Latency	Egy request mennyi ideig tart. Ne csak average, nezd p95/p99-et is.
p95 / p99	p95: requestek 95%-a ennel gyorsabb. p99: 99%-a ennel gyorsabb. Tail latency.
Concurrency	Egyszerre futo in-flight requestek. Durva keplet: rps x latency seconds.
I/O	Input/Output. DB, API, file, Redis, queue, network. Node.js I/O-bound munkara eros.
CPU-bound	A CPU szamol sokat: hashing, image/video, nagy JSON, heavy loop. Event loopot blokkolhat.
WebSocket vs HTTP	WebSocket: persistent, bidirectional, kis frame overhead. HTTP/2/3 is reuse/multiplex.
gRPC	HTTP/2 + Protobuf. Internal service-to-service contract, streaming, codegen.
VM vs container	VM sajat OS-sel nehezebb. Container host kernelt hasznal, gyorsabb indulas/scaling.
Readiness probe	Kaphat-e trafficet az instance? App futasa nem eleg, lehet DB/config/warmup meg nincs kesz.
Liveness probe	El-e a process, restart kell-e? Ne legyen tul szigoru dependency check.
/health vs /ready	/health tul altalanos. /live = el-e. /ready = fogadhat-e trafficet.

2. Direction, dependencies, architecture

Fogalom	Emlekezteto
Upstream	Aki teged hiv. Pl. frontend, mobile app, partner system -> Your API.
Downstream	Amit te hivsz. Pl. DB, payment provider, CRM, email, partner API.
Outbound	Kifele meno hivas a service-edbol egy dependency fele.
Inbound	Bejovo request vagy event a service-edbe.
Coupling	Mennyire fugg ket rendszer egymastol. Loose: contract. Tight: belso DB/workflow ismerete.
Integration fan-out	Egy input utan sok downstream hivas/event. Kritikus sync, mellekes async.
Coarse-grained service	Kevesebb, nagyobb uzleti service. Nem tul apro microservice-ek.
SOA	Service-oriented architecture: uzleti capability-k service-ekbe szervezve, contractokkal.
ESB	Enterprise Service Bus: central integration layer routingra, transformra, protocol mediationre.
SOA vs ESB	SOA = architecture style. ESB = gyakori technikai integration backbone SOA alatt.
API Gateway	Edge layer: routing, auth, rate limit, TLS, logging. Idealisan vekony.
ESB bottleneck	Sok transform/workflow logikat es csapatfuggest gyujthet ossze.
Node.js alapu SOA	Igen. Node lehet service implementacio. Node maga nem ESB termek.

3. Resilience and overload

Fogalom	Emlekezteto
Timeout	Ne varj orokke downstreamre. Minden outbound callnak legyen limitje.
Retry	Transient hibara jo, de limit, backoff + jitter kell. Non-idempotent muveletnel veszelyes.
Circuit breaker	Beteg downstreamre ideiglenesen fail fast. Closed -> open -> half-open.
Graceful degradation	Nem kritikus feature kieshet, de core flow menjen tovabb. Pl. email/loyalty kesobbre.
Bulkhead	Eroforras-izolacio. Egy lassu dependency ne egye meg az osszes connectiont/concurrencyt.
Per-downstream concurrency	Dependency-nkent max parhuzamos hivas. Pl. CRM max 10, payment max 50.
Backpressure	Tul sok munka jon, lassitani/visszautasitani kell. 429, 503 Retry-After, queue limit.
Rate limit	Idoegyseg alatti request limit. Pl. 1000/min per API key. Abuse es overload ellen.
Stale data	Elavult adat cachebol/read replicabol/event delaybol. Critical flow elott revalidate.
Retry storm	Sok kliens egyszerre retry-zik, tovabb rontva az incidenst. Jitter/circuit breaker kell.
CRM	Customer Relationship Management. Customer profile, history, marketing/support adatok.

4. Integration patterns

Fogalom	Emlekezteto
Webhook	HTTP callback eventekhez. Altalaban elore regisztralt URL, provider POST-ol eventet.
Webhook vs pub/sub	Webhook = HTTP delivery kulso rendszerekhez. Pub/sub broker = Kafka/RabbitMQ/Service Bus.
Per-request callback URL	Nem allando webhook. Egy hosszu async muvelethez adott callback URL.
Retention	Meddig orzi meg a broker az eventet/message-et. Consumer downtime utan catch-uphoz kell.
Aggregator	Tobb downstream valaszbol egy egyseges valaszt epit. Timeout/partial/error szabaly kell.
Scatter-gather	Szetkuldod tobb forrashoz, majd osszegyujtod az eredmenyt.
Choreography	Kozponti vezerlo nelkul eventekre reagalnak a service-ek. Loose coupling, nehezebb tracing.
Orchestration	Egy coordinator vezerli a lepeseket. Atlathatobb flow, de bottleneck/coupling lehet.
Saga	Tobb local transaction + compensation distributed transaction helyett.
Compensation	Uzleti visszacsinolo muvelet. Pl. ticketing fail utan refund + cancel booking.
Outbox pattern	Business DB valtozas es event rekord mentese ugyanabban a transactionben. Worker publishol.
Idempotent consumer	Ugyanaz az event tobbszor johet, a vegeredmeny megse duplazodjon.

5. Data, DB, consistency

Fogalom	Emlekezteto
Local transaction	Egy DB transaction boundaryn beluli ACID muvelet. Kozos DB-s monolitnal is mukodik.
Distributed consistency	Tobb service/DB/external provider kozott nincs egyszeru kozos COMMIT.
ACID	Atomicity, Consistency, Isolation, Durability. DB transaction garanciak.
Atomicity	Minden vagy semmi. Ha egy lepes fail, rollback.
Consistency	Ervenyes allapotbol ervenyes allapotba kerul az adat.
Isolation	Parhuzamos transactionok ne lassanak rossz felkesz allapotot.
Durability	COMMIT utan az adat tuleli a crasht/restartot.
DB constraint	DB altal betartatott szabaly: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK.
Lock	DB ideiglenesen lefoglal sort/tablat consistency miatt. Mas transaction varhat.
Deadlock	Ket transaction korben egymas lockjara var. DB az egyiket rollbackeli.
Lock wait	Query nem szamol, hanem lock felszabadulasara var. API kivulrol lassunak latszik.
SQL optimization	Execution plan, index, full scan, joins, pagination, N+1, pool wait, locks.
Reconciliation	Utólag osszeveted rendszerek allapotat, es javitod/jelzed az elterest.

6. API security terms

Fogalom	Emlekezteto
Nonce	Egyszer hasznalatos random ertek replay attack ellen.
Replay protection	Regi, de ervenyes requestet ne lehessen ujrajatszani. Signature + timestamp + nonce/eventId.
Revocation	Token/API key/permission visszavonasa lejarat elott.
JWT revocation issue	Self-contained JWT expiry-ig valid lehet. Short access token, refresh revocation, denylist.
Signature validation	Webhooknal ellenorzod, hogy a provider kuldte, es a payload nem modosult.
Timestamp check	Webhook/request ne legyen tul regi. Replay vedelem resze.
401 vs 403	401 = nem vagy hitelesitve. 403 = tudom ki vagy, de nincs jogod.
Opaque token	Token tartalmat a service nem olvassa, auth server/introspection mondja meg, valid-e.
mTLS	Kliens es szerver is certificate-tel igazolja magat. Eros service-to-service security.

30 masodperces checklist

Ezeket a mondatokat/iranyokat erdemes fejben tartani interju elott.

Tipikus helyzetek

Fogalom	Emlekezteto
Sizing	100M/month = kb. 40 rps avg. Mindig peak, request complexity, p95/p99, downstream limit.
API slow	RED: rate/errors/duration. Aztan USE: CPU, memory, DB pool, queue, event loop.
Downstream slow	Timeout -> retry backoff+jitter -> circuit breaker -> fallback/degraded mode.
Fan-out	Critical path legyen rovid. Nem kritikus ertesites async event/queue.
Distributed data	Ne distributed transaction legyen default. Saga, outbox, idempotency, reconciliation.
Node.js	Eros I/O-heavy integration service-re. CPU-heavy kodot ne blokkolva futtasd.
DB issue	Execution plan, index, full scan, N+1, locks/deadlocks, pool wait.
Security	AuthN/AuthZ, 401/403, token validation, revocation, signature, nonce, replay protection.
Container	Stateless, horizontal scaling, CPU/mem limit, readiness/liveness, graceful shutdown.
Architecture	Boundary, coupling, team ownership, ops cost, deployment independence dont.

Mini angol mondat

I would not size or design the system from the monthly request number alone. I would look at peak traffic, request complexity, downstream dependencies, database limits, and p95/p99 latency, then validate the design with realistic load tests.